VISHVKISAN

Sample Data and Software Test Cases

Purpose: This document provides sample records and test cases for checking the VishvKisan prototype. All farmer names, phone numbers, prices, weather readings, and orders below are fictional or illustrative demo data.

1. Sample Data

A. Farmer accounts

ID	Sample farmer	Location	Language	Demo status
FARM-001	Ranjit Das	Sualkuchi, Kamrup	Assamese	Pending
FARM-002	Mina Bora	Hajo, Kamrup	English	Verified (demo only)

B. Produce listings

Product ID	Product	Price/kg	Stock	Test purpose
PROD-001	Assam Local Rice	■48	120 kg	Available listing
PROD-002	Seasonal Vegetables	■35	40 kg	Available listing
PROD-003	Mustard Greens	■25	0 kg	Out-of-stock case

C. Other demo records

- Market-price examples: Rice ■32/kg and tomato ■28/kg in a fictional “Demo local market”. These are not live mandi prices.
- Weather example: Sualkuchi, 28°C, partly cloudy, 72% humidity, 30% rainfall chance. This is mock data, not a forecast.
- Sample order ORD-1001: 2 kg rice × ■48/kg = ■96; status “placed”, demo-unpaid.
- Government scheme cards may link to official portals, but the prototype does not itself certify eligibility or verification.

2. Test Cases

TC ID	Feature	Test steps / input	Expected result	Priority
TC-01	Marketplace	Open customer marketplace.	Product cards and key details are visible.	High
TC-02	Search	Search for “rice”.	Matching rice listing appears; unrelated items are filtered.	High
TC-03	Category filter	Choose “Vegetables”.	Only vegetable listings are shown.	Medium
TC-04	Price display	Open PROD-001.	Price is shown as ■48 per kg; no misleading unit.	High
TC-05	Valid order	Order 2 kg of PROD-001.	Order total is ■96 before any separately disclosed fees.	High
TC-06	Stock limit	Try to order 121 kg of PROD-001 (stock 120 kg).	Order is blocked with a clear stock message.	High
TC-07	Out of stock	Try to order PROD-003 (stock 0 kg).	Order cannot be placed.	High
TC-08	Invalid quantity	Enter 0, a negative value, or text as quantity.	Input is rejected and a clear validation message appears.	High
TC-09	Farmer listing	Enter product name, price, unit and stock.	Valid listing can be previewed/saved in the prototype.	High
TC-10	Invalid price	Enter a negative price.	Listing is rejected; price must be zero or greater.	High
TC-11	Order tracking	Open a demo order and inspect its status.	Status label matches the stored demo order.	Medium
TC-12	Language selector	Change the language option.	Supported interface text changes or an honest “not supported yet” message appears.	Medium
TC-13	Responsive layout	Check on desktop and narrow mobile width.	Text and controls remain readable; no major horizontal overflow.	High

TC ID	Feature	Test steps / input	Expected result	Priority
TC-14	Scheme information	Open PM-KISAN card and follow official link.	Official portal opens; app does not falsely guarantee eligibility.	Medium
TC-15	Demo-data labeling	Inspect market and weather panels.	Mock data is labelled demo/sample; it is not presented as live data.	High
TC-16	Login/authentication	Try invalid credentials or an unverified OTP flow.	Access is not falsely reported as securely authenticated; demo limitations are clear.	High
TC-17	Error handling	Simulate missing product fields or unavailable data.	Page remains usable and displays a clear fallback/error.	Medium
TC-18	Farmer verification	Open a demo-verified farmer profile.	Demo badge is clearly marked as sample; no false claim of official verification.	High

3. Example Calculation Code

Use this simple JavaScript example to validate a sample order total. It is an example, not an automatic integration with your current website.

```
export function calculateOrderTotal(pricePerKg, quantityKg) {
  if (!Number.isFinite(pricePerKg) ||
      !Number.isFinite(quantityKg)) {
    throw new TypeError("Price and quantity must be numbers.");
  }
  if (pricePerKg < 0 || quantityKg <= 0) {
    throw new RangeError(
      "Price cannot be negative and quantity must be greater than zero."
    );
  }
  return Number((pricePerKg * quantityKg).toFixed(2));
}

// Example:
console.log(calculateOrderTotal(48, 2)); // 96
```

4. Running the automated tests

The accompanying JavaScript file contains sample records, helper functions, and Node.js test cases. Because the file uses JavaScript module syntax and Node's built-in test runner, save it with an .mjs extension if your project does not already use ES modules, then run:

```
node --test VishvKisan_Sample_Data_and_Test_Cases.mjs
```

5. Test execution record (fill in after testing)

TC ID / range	Actual result	Status	Tester / date
TC-01 to TC-06	_____	Pass / Fail	_____
TC-07 to TC-12	_____	Pass / Fail	_____
TC-13 to TC-18	_____	Pass / Fail	_____

Important limitations: The supplied sample records are fictional and should be used only for demonstrations and testing. This document does not claim that the tests have been executed against your deployed website. Record actual outcomes after manually running each test. Real authentication, payments, inventory persistence, official farmer verification, live weather, and live mandi prices must be tested separately if those integrations are implemented.